Distributed Logging - Part4

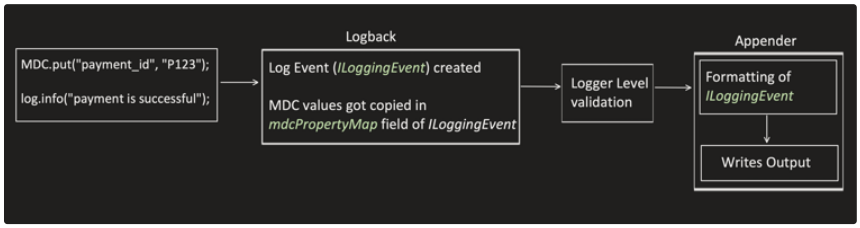
MDC (Mapped Diagnostic Context):

In logging framework, this plays a very critical role.

- Its **per-thread** key-value map.

  MDC.put("payment_id", "P123");

- Logging framework (Logback, Log4j2) can automatically read and attach the key-value to log events.



In Pattern encoder, by-default MDC fields access is false, we explicitly need to use %X if we want to use the MDC data.

Concept && Coding

%X is to fetch all MDC data

Key1=value1, key2=value2 etc.

```
<appender name="CONSOLE_WITH_ALL_MDC"
class="ch.qos.logback.core.ConsoleAppender">
    <encoder>
        <pattern>%d %level %logger - %msg - %X %n</pattern>
    </encoder>
</appender>

```

%X{key} is to fetch specific MDC key value only
(pls note: only value for the specific key comes)

```
<appender name="CONSOLE_WITH_MDC"
class="ch.qos.logback.core.ConsoleAppender">
    <encoder>
        <pattern>%d %level %logger - %msg - %X{payment_id} %n</pattern>
    </encoder>
</appender>

```

In Structured (JSON) encoder, by-default MDC fields access is true.

all MDC data will be fetched and added in existing json log

Key1 : value1, key2 : value2 etc.

```
<appender name="CONSOLE_WITH_JSON" class="ch.qos.logback.core.ConsoleAppender">
    <encoder class="net.logstash.logback.encoder.LogstashEncoder">
        <includeMdc>true</includeMdc> <!--by-default is true -->
    </encoder>
</appender>

```

Let's see an example:

```
@RestController
public class PaymentController {

    Logger log = LoggerFactory.getLogger(PaymentController.class);

    @GetMapping("/payments")
    public String getPayments() {
/*
      Its a key-value map
 */
        MDC.put("payment_id", "P123");
        MDC.put("user_id", "U1234");
      /*
      Logging framework, automatically adds MDC data with each log statement.
    */
        log.info("payment is successful");
        MDC.clear();

        return "successfully fetched all payments";
    }
}

```

Concept && Coding

logback-spring.xml

```
<appender name="CONSOLE_WITH_MDC" class="ch.qos.logback.core.ConsoleAppender">
    <encoder>
        <pattern>%d %level %logger - %msg - %X{payment_id}%n</pattern>
    </encoder>
</appender>

<appender name="CONSOLE_WITH_ALL_MDC" class="ch.qos.logback.core.ConsoleAppender">
    <encoder>
        <pattern>%d %level %logger - %msg - %X%n</pattern>
    </encoder>
</appender>

<appender name="CONSOLE_WITH_JSON" class="ch.qos.logback.core.ConsoleAppender">
    <!-- JSON encoder -->
    <encoder class="net.logstash.logback.encoder.LogstashEncoder">
        <includeMdc>true</includeMdc> <!--by-default is true only -->
    </encoder>
</appender>
```

```
<!-- Specific logger config -->
<logger name="com.concepts.PaymentController" level="INFO" additivity="false">
    <appender-ref ref="CONSOLE_WITH_MDC"/>
    <appender-ref ref="CONSOLE_WITH_ALL_MDC"/>
    <appender-ref ref="CONSOLE_WITH_JSON"/>
</logger>
```

Output:

```
2025-12-19 15:15:49,245 INFO org.springframework.boot.web.embedded.tomcat.TomcatWebServer - Tomcat started on port 8080 (http) with context path '/'
2025-12-19 15:15:49,249 INFO com.concepts.Logger1Application - Started Logger1Application in 0.662 seconds (process running for 0.784)
2025-12-19 15:15:51,367 INFO org.apache.catalina.core.ContainerBase.[Tomcat].[localhost].[/] - Initializing Spring DispatcherServlet 'dispatcherServlet'
2025-12-19 15:15:51,367 INFO org.springframework.web.servlet.DispatcherServlet - Initializing Servlet 'dispatcherServlet'
2025-12-19 15:15:51,368 INFO org.springframework.web.servlet.DispatcherServlet - Completed initialization in 1 ms
2025-12-19 15:15:51,381 INFO com.concepts.PaymentController - payment is successful - P123
2025-12-19 15:15:51,381 INFO com.concepts.PaymentController - payment is successful - user_id=U1234, payment_id=P123
{"@timestamp":"2025-12-19T15:15:51.381113+05:30","@version":"1","message":"payment is successful","logger_name":"com.concepts.PaymentController","thread_name":"http-nio-8080-exec-1","level":"INFO","level_value":20000,"user_id":"U1234","payment_id":"P123"}
```

But you might be thinking, why are you doing **MDC.clear():**

```
@RestController
public class PaymentController {

    Logger log = LoggerFactory.getLogger(PaymentController.class);

    @GetMapping("/payments")
    public String getPayments() {

        MDC.put("payment_id", "P123");
        MDC.put("user_id", "U1234");
        log.info("payment is successful");
        MDC.clear();

        return "successfully fetched all payments";
    }
}
```

Concept && Coding

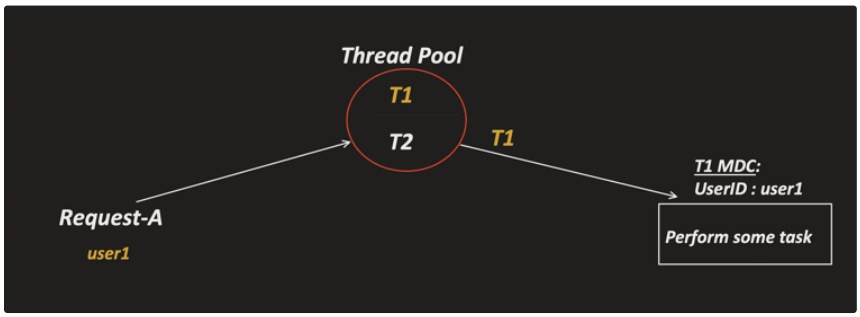
This is done to avoid **LOG POLLUTION**, how?

- As we know, Its **per-thread** key-value map.
- Means each thread own exactly one MDC map.

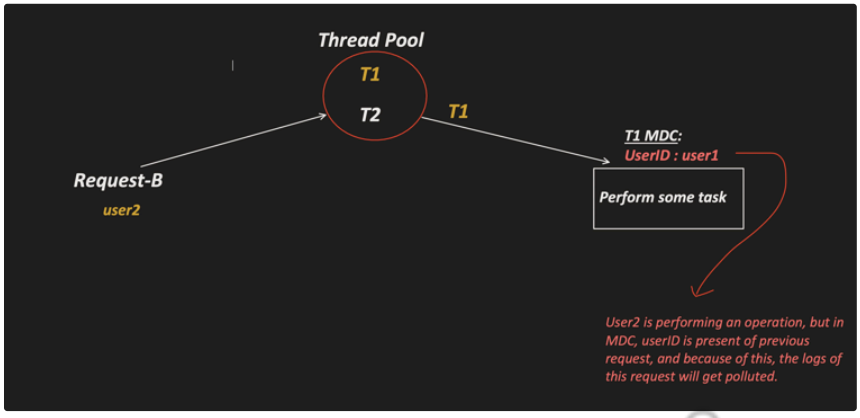
But now-a-days because of ***ThreadPoolExecutor***, same thread can be re-used for multiple requests.

So, If thread MDC is not cleared for Request-A, and same thread picks the other request Request-B, the withing Request-B it can see the previous request Request-A data, which cause **LOG POLLUTION.**

Log Pollution Example:



Request-A completed, and Thread T1 is put back again in Thread Pool, but its MDC is not cleared.



Concept && Coding

Another important point to notice:

- Whenever we create a new thread like via @Async. Your parent thread MDC is not passed to child thread.

Again, every thread has their own MDC.

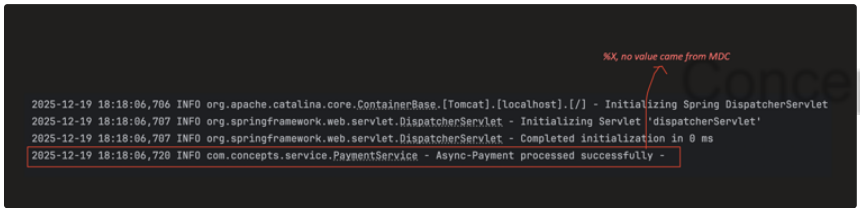
```
@RestController
public class PaymentController {

    @Autowired
    PaymentService paymentService;

    @GetMapping("/payments/async")
    public String getPaymentsAsync() {

        MDC.put("payment_id", "P123");
        MDC.put("user_id", "U1234");
        paymentService.processPayment();
        MDC.clear();

        return "successfully fetched all payments";
    }
}

```

```
@Component
public class PaymentService {

```

```java
    Logger log = LoggerFactory.getLogger(PaymentService.class);

    @Async
    public void processPayment() {
      /*
      Child thread, this thread has its own MDC
      */

        log.info("Async-Payment processed successfully");
    }
}

```

logback-spring.xml

```xml
<appender name="CONSOLE_WITH_ALL_MDC" class="ch.qos.logback.core.ConsoleAppender">
    <encoder>
        <pattern>%d %level %logger - %msg - %X%n</pattern>
    </encoder>
</appender>

<logger name="com.concepts.service.PaymentService" level="INFO" additivity="false">
    <appender-ref ref="CONSOLE_WITH_ALL_MDC"/>
</logger>

```



Concept && Coding

So, how to copy the Parent MDC value to child (new Thread) MDC?

Best way is to use: **TaskDecorator**

Pls Note:

TaskDecorator is a feature of Spring framework. Nothing to do with Logging framework.

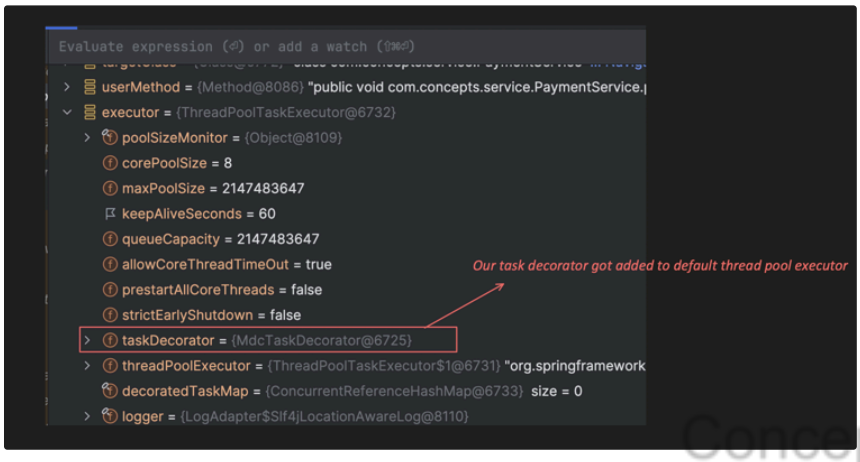
TaskDecorator is just a wrapper around the Runnable task, lets understand with below flow:

```java
@Component
public class MdcTaskDecorator implements TaskDecorator {

    @Override
    public Runnable decorate(Runnable runnable) {

        Map<String, String> contextMap = MDC.getCopyOfContextMap();

        return () -> {
            if (contextMap != null) {
                MDC.setContextMap(contextMap);
```

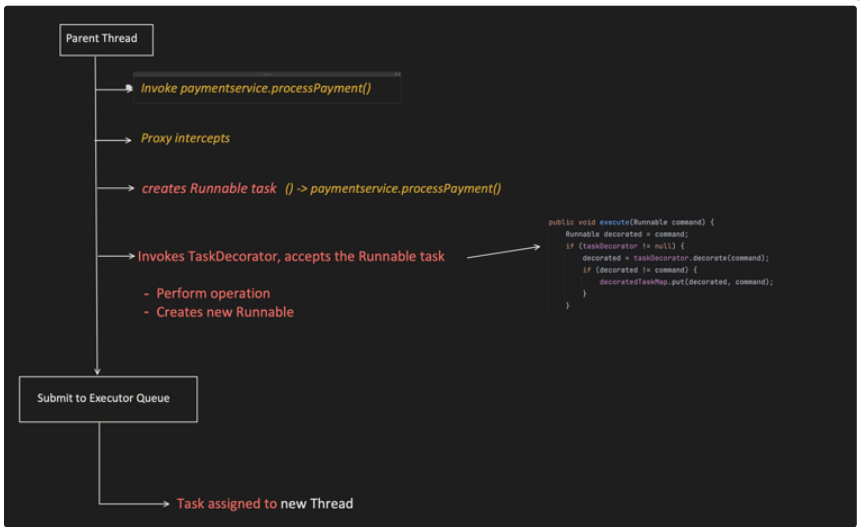
```
            }
            try {
                runnable.run();
            } finally {
                MDC.clear();
            }
        };
    }
}

```

- When we create the TaskDecorator, we can associate it with any ThreadPoolExecutor.

- If we don't have any custom ThreadPoolExecutor, this TaskDecorator will get added to default executor itself.



Concept && Coding



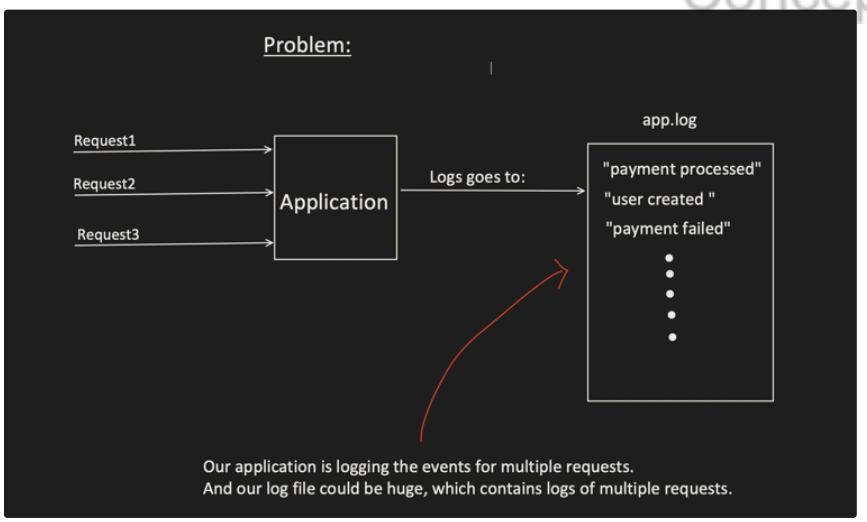

```
@Component
public class MdcTaskDecorator implements TaskDecorator {

    @Override
    public Runnable decorate(Runnable runnable) {
      /*
      Currently we are in Parent Thread only,
      so read the values
        */

        Map<String, String> contextMap = MDC.getCopyOfContextMap();
```

```java
      /*
      New runnable is just a wrapper,

      - set the MDC with values of the Parent thread MDC
        - Actual runnable
        - MDC clear
        So we just wrapped the actual runnable

     */

        return () -> {
            if (contextMap != null) {
                MDC.setContextMap(contextMap);
            }
            try {
                runnable.run();
            } finally {
                MDC.clear();
            }
        };
    }
}
```

```
2025-12-19 19:59:32,268 INFO com.concepts.Logger1Application - Started Logger1Application in 0.761 seconds (process running for 0.886)
2025-12-19 19:59:33,548 INFO org.apache.catalina.core.ContainerBase.[Tomcat].[localhost].[/] - Initializing Spring DispatcherServlet 'dispatcherServlet'
2025-12-19 19:59:33,548 INFO org.springframework.web.servlet.DispatcherServlet - Initializing Servlet 'dispatcherServlet'
2025-12-19 19:59:33,549 INFO org.springframework.web.servlet.DispatcherServlet - Completed initialization in 1 ms
2025-12-19 19:59:38,570 INFO com.concepts.service.PaymentService - Async-Payment processed successfully - user_id=U1234, payment_id=P123
```

Now we know what is **MDC**: Next we need to understand what is **Correlation id**

Concept && Coding



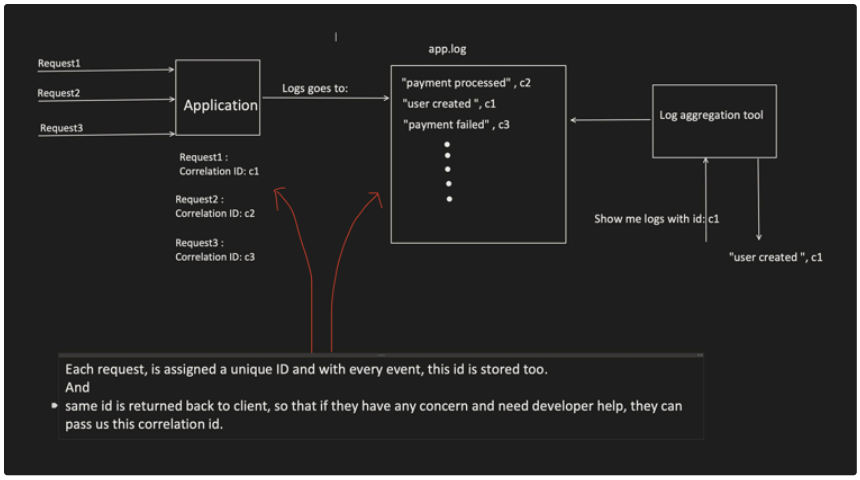
How would you find the logs for specific request say Request1?

Correlation id, helps to resolve this

- It is nothing but a unique identifier for a single request.

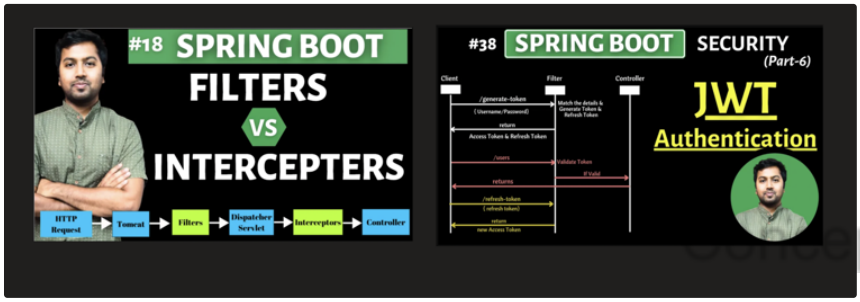
  And

- The same ID is attached with every log event. (generally saved in MDC, so that every event get this ID too)



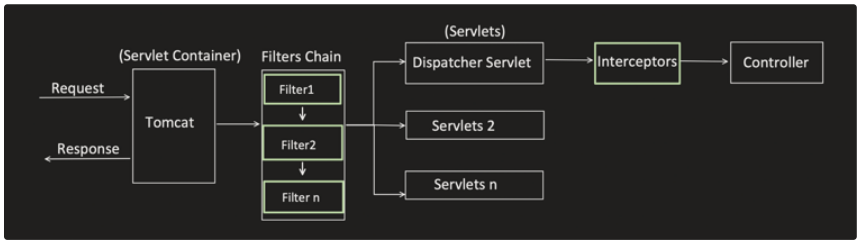
Pre-requisite: Filters

In Spring boot series, we have covered FILTERS in depth, also in JWT implementation, we have used Filters a lot, so more in-depth can be found in that too.



Filter:

Is used when we want to intercept HTTP Request and Response.




```java
@RestController
public class PaymentController {

    Logger log = LoggerFactory.getLogger(PaymentController.class);

    @GetMapping("/payments/correlation")
    public String getPaymentsWithCorrelationId() {

        log.info("payment is successful");

        return "successfully fetched all payments";
    }
}

```

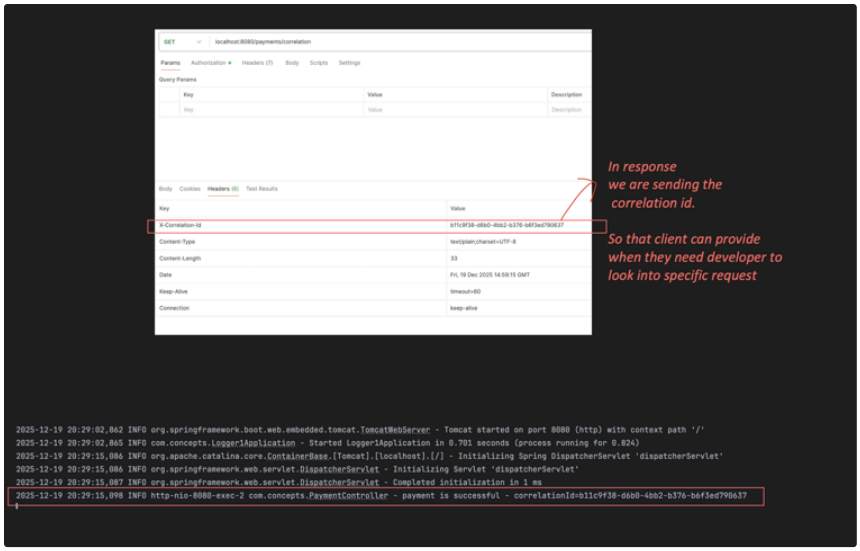
logback-spring.xml

```
<appender name="CONSOLE_WITH_ALL_MDC" class="ch.qos.logback.core.ConsoleAppender">
    <encoder>
        <pattern>%d %level %thread %logger - %msg - %X %n</pattern>
    </encoder>
</appender>

<!-- Specific logger config -->
<logger name="com.concepts.PaymentController" level="INFO" additivity="false">
    <appender-ref ref="CONSOLE_WITH_ALL_MDC"/>
</logger>

```

Our Custom Filter:

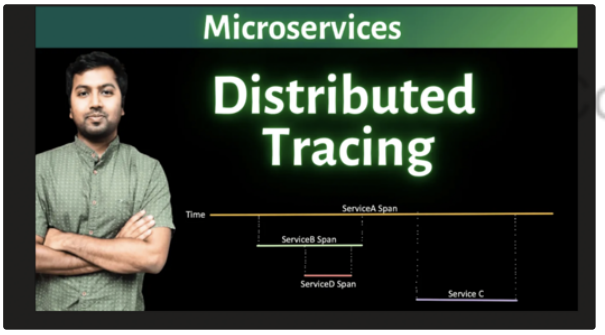
```
@Component
public class CorrelationIdFilter extends OncePerRequestFilter {

    public static final String CORRELATION_ID = "correlationId";
    public static final String CORRELATION_HEADER = "X-Correlation-Id";

    @Override
    protected void doFilterInternal(HttpServletRequest request,
                                    HttpServletResponse response,
                                    FilterChain filterChain)
            throws ServletException, IOException {

        String correlationId = request.getHeader(CORRELATION_HEADER);

        // if there is no correlation id coming from request, set a random one
        if (correlationId == null || correlationId.isBlank()) {
            correlationId = UUID.randomUUID().toString();
        }

        try {
            MDC.put(CORRELATION_ID, correlationId);
            response.setHeader(CORRELATION_HEADER, correlationId);
            filterChain.doFilter(request, response);
        } finally {
            MDC.clear();
        }
    }
}

```

Concept && Coding



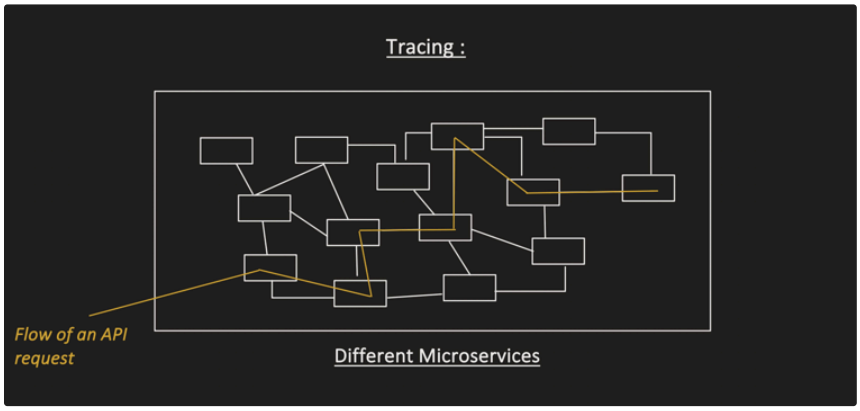
Now the final part, how to achieve Distributed Logging:

Pre-requisite: Distributed Tracing



oncept && Coding

Why we need Distributed Tracing? and what's the difference between TraceID & Correlation ID?



- Tracing tracks the path of a request across microservices.
- For each Request, unique Trace ID is created at Entry point.

- Same Trace ID is passed to different microservices, when they are invoked.

- So, we get to know:
  - Which service called which service.
  - How much time each service took.
  - Status code (200, 400, 500) of each service.
    etc.

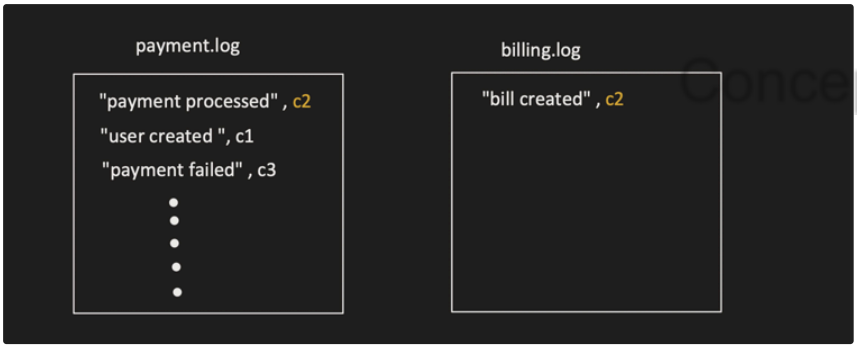
- But it does NOT store detailed logs.

Logging and Correlation ID:

- As part of Logging, details logs about the request is stored.

- To filter the logs of specific request, each request is assigned unique id i.e Correlation ID.

- But it does not stich or shows which service calls which service or in simple term: What all services got invoked during the lifecycle of a particular request.



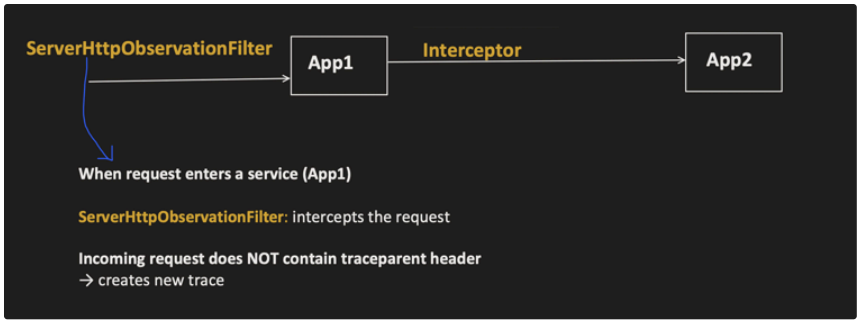
Concept && Coding

No matter if 10 services got invoked, all 10 services should log event for particular request with same correlation id

Now, one question might come to your mind, both Trace-ID and Correlation-ID are unique for the request. Can they be same?

Answer is YES, and that's what many big tech companies does.

If we can recall from Distributed Tracing topic, it make use of "*ServerHttpObservationFilter*"



And interesting part is, this filter also adds "**traceID**" in MDC.

So this unique trace id, will also get appended with every log event.

Advantage:

- We don't need to add custom filter to add correlation-id in MDC, already unique trace is present in MDC.
- We don't need to pass the correlation id in request header when other service is getting invoked. Because Trace id is already passed in the header to downstream services.

Concept && Coding

But still, we can add Filter for correlation id, and re-use the Trace id if present instead of computation of new UUID:

- we can send correlation id in response header.
- What if tracing is disabled, correlation id can be independent.

Tracing dependencies:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>

<dependency>
    <groupId>io.micrometer</groupId>
    <artifactId>micrometer-tracing-bridge-otel</artifactId>
</dependency>

<dependency>
    <groupId>io.opentelemetry</groupId>
    <artifactId>opentelemetry-exporter-otlp</artifactId>
</dependency>

```

```
server.port=8080
spring.application.name=logging-App-1

# Micrometer tracing
management.tracing.sampling.probability=1.0

# OTLP exporter (Jaeger)
management.otlp.tracing.endpoint=http://localhost:4318/v1/traces
```

```

```

```java
@RestController
public class PaymentController {

    Logger log = LoggerFactory.getLogger(PaymentController.class);

    @GetMapping("/payments/correlation")
    public String getPaymentsWithCorrelationId() {

        log.info("payment is successful");

        return "successfully fetched all payments";
    }
}

```

```java
@Component
public class CorrelationIdFilter extends OncePerRequestFilter {

    public static final String CORRELATION_ID = "correlationId";
    public static final String CORRELATION_HEADER = "X-Correlation-Id";

    @Autowired
    private Tracer tracer;

    @Override
    protected void doFilterInternal(HttpServletRequest request,
                                    HttpServletResponse response,
                                    FilterChain filterChain)
            throws ServletException, IOException {

        try {
          /*
          First priority to trace id, if its empty, then only generates
          new Correlation id else reuse trace id
          */
            String uniqueID = tracer.currentSpan().context().traceId();
            if (uniqueID == null) {
                uniqueID = UUID.randomUUID().toString();
            }
        /*

        Put in MDC, now our MDC has trace id and correlation id both

          */
            MDC.put(CORRELATION_ID, uniqueID);
          /*
          In response also we added
          */
            response.setHeader(CORRELATION_HEADER, uniqueID);
            filterChain.doFilter(request, response);
        } finally {
          /*
          Need to clear the MDC, so that this thread data does not leak
          to other request.
          */
            MDC.clear();
        }
    }
}

```

Concept && Coding

```
<appender name="CONSOLE_WITH_ALL_MDC" class="ch.qos.logback.core.ConsoleAppender">
    <encoder>
```

```
        <pattern>%d %level %thread %logger - %msg - %X %n</pattern>
    </encoder>
</appender>

<!-- Specific logger config -->
<logger name="com.concepts.PaymentController" level="INFO" additivity="false">
    <appender-ref ref="CONSOLE_WITH_ALL_MDC"/>
</logger>

```

```
2025-12-19 21:39:43,521 INFO org.springframework.boot.web.embedded.tomcat.TomcatWebServer - Tomcat started on port 8080 (http) with context path '/'
2025-12-19 21:39:43,531 INFO com.concepts.Logger1Application - Started Logger1Application in 1.043 seconds (process running for 1.177)
2025-12-19 21:39:49,202 INFO org.apache.catalina.core.ContainerBase.[Tomcat].[localhost].[/] - Initializing Spring DispatcherServlet 'dispatcherServlet'
2025-12-19 21:39:49,202 INFO org.springframework.web.servlet.DispatcherServlet - Initializing Servlet 'dispatcherServlet'
2025-12-19 21:39:49,203 INFO org.springframework.web.servlet.DispatcherServlet - Completed initialization in 1 ms
2025-12-19 21:39:49,226 INFO http-nio-8080-exec-2 com.concepts.PaymentController - payment is successful - traceId=9e7d21299f4ea8a1e6f22c0ca781c9aa, spanId=2c3655fc800de28b, correlationId=9e7d21299f4ea8a1e6f22c0ca781c9aa
```

Because of Tracing, other service also has the same filter, which uses the same Trace id and set in MDC. And in that application also, if required we can add Correlation id Filter.

Concept && Coding